CNN Compression

Sparsh Mittal

“Deep Compression” Technique
A Model-Size Aware CNN-Compression Technique

Sparsh Mittal

CNN = Convolutional Neural Network

Motivation for CNN Compression

Growing network sizes:
LeNet-5 (1M parameters), DeepFace (120M), AlexNet (60M), VGG-16 (130M), Coates (10B)

Reducing storage requirement offers many benefits:

1. Model can fit in cache; reduces memory bottleneck

2. Results in a small executable (apps> 100MB are downloaded over WiFi only)

3. Reduces the power for inference (esp. useful for mobiles where power budget is few watts)

Power/Energy Benefits

Table I
ENERGY TABLE FOR 45NM CMOS PROCESS [9]. DRAM ACCESS USES THREE ORDERS OF MAGNITUDE MORE ENERGY THAN SIMPLE ARITHMETIC AND 128X MORE THAN SRAM.

Operation	Energy [pJ]	Relative Cost
32 bit int ADD	0.1	1
32 bit float ADD	0.9	9
32 bit int MULT	3.1	31
32 bit float MULT	3.7	37
32 bit 32KB SRAM	5	50
32 bit DRAM	**640**	**6400**

Running a billion-parameter network at 20 frames/sec would consume power = 20 x 1G x 640 = 12.8 W
Would also need a well-provisioned memory system

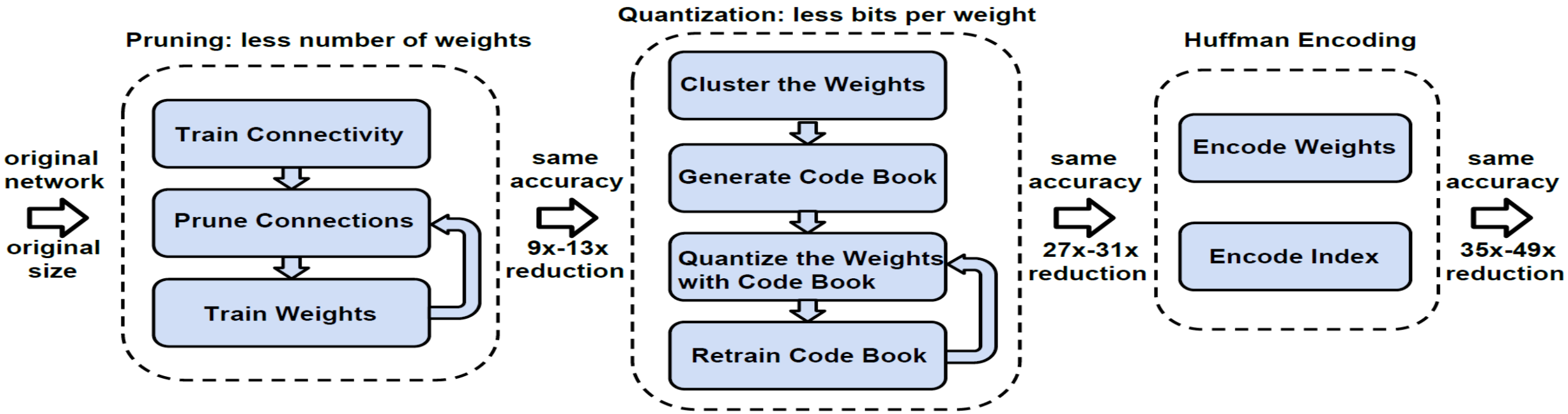
Image source: Mark Horowitz. Energy table for 45nm process, Stanford VLSI wiki

Deep Compression Uses Three Ideas

- Prune network by removing the redundant connections, keeping only the most informative connections.
- Quantize weights so that multiple connections share the same weight, thus only the codebook (effective weights) and the indices need to be stored.
- Apply Huffman coding to take advantage of the biased distribution of effective weights.

Reference: "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding", S. Han et al., ICLR, 2016

Block Diagram



The compression rate includes the meta-data for sparse representation.

Han, S., Mao, H. and Dally, W.J., 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*.

First Compression Technique:

Pruning

Synapse Pruning in Human Brain Development

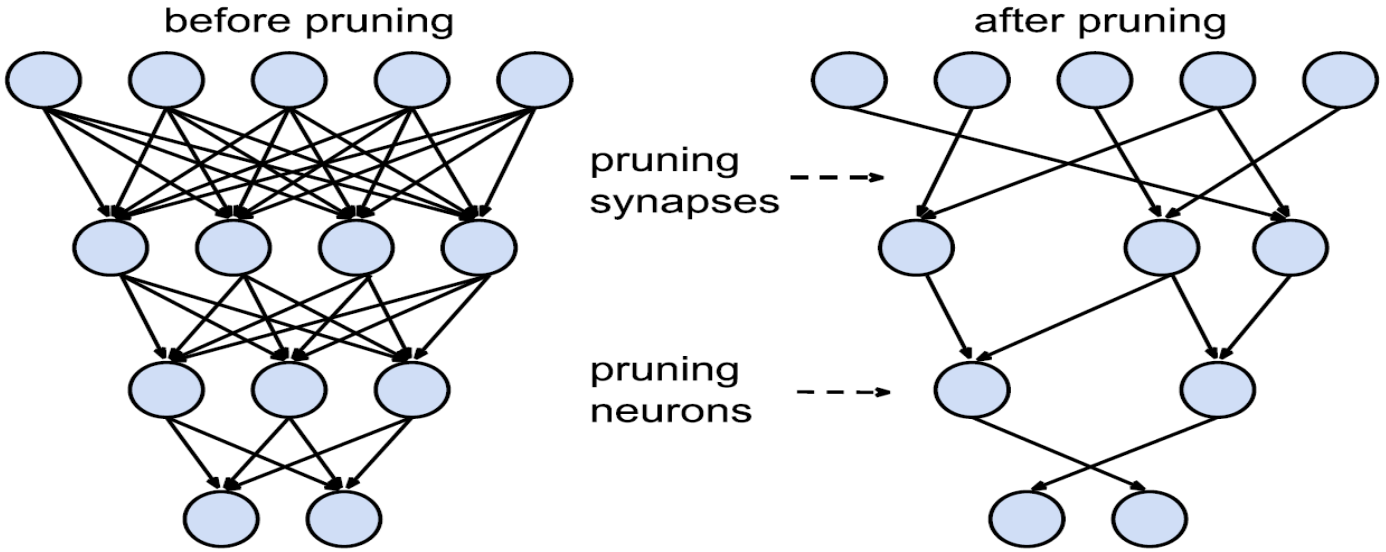
This process is biologically inspired since even adolescent brain prunes neurons and connections.

Age	Number of Connections	Stage
at birth	50 Trillion	newly formed
1 year old	1000 Trillion	peak
10 year old	500 Trillion	pruned and stabilized

Walsh, Nature 2013.

Pruning

- Remove the weights below a threshold.
- Retrain network to learn the final weights for the remaining connections.

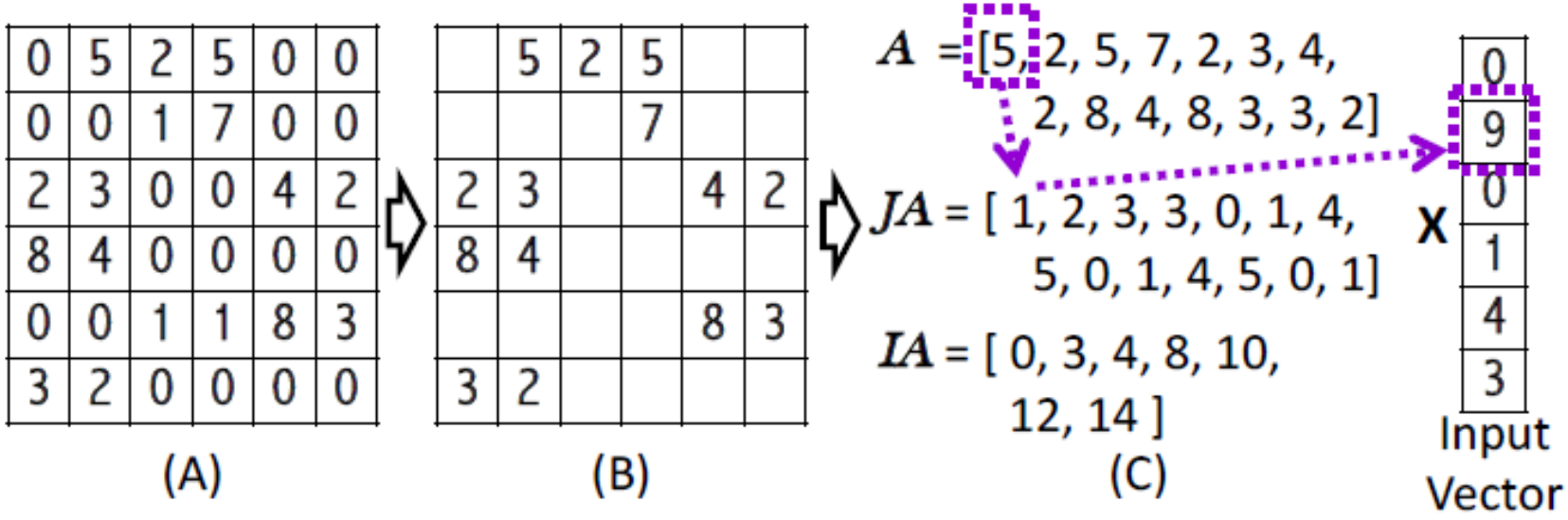


Some neurons can disappear if all their inputs are pruned

Example: for a centered image, weights dealing with peripheral pixels are ~zero and can be pruned.

Han, Song, et al. "Learning both weights and connections for efficient neural networks." *arXiv:1506.02626* (2015).

Example of pruning



- (A) Dense weight matrix
- (B) Sparse weight matrix;
- (C) Compressed Sparse Rows (CSR) format for sparse matrix in (B)

Weight Sharing and Quantization

Second Compression Technique

Weight sharing

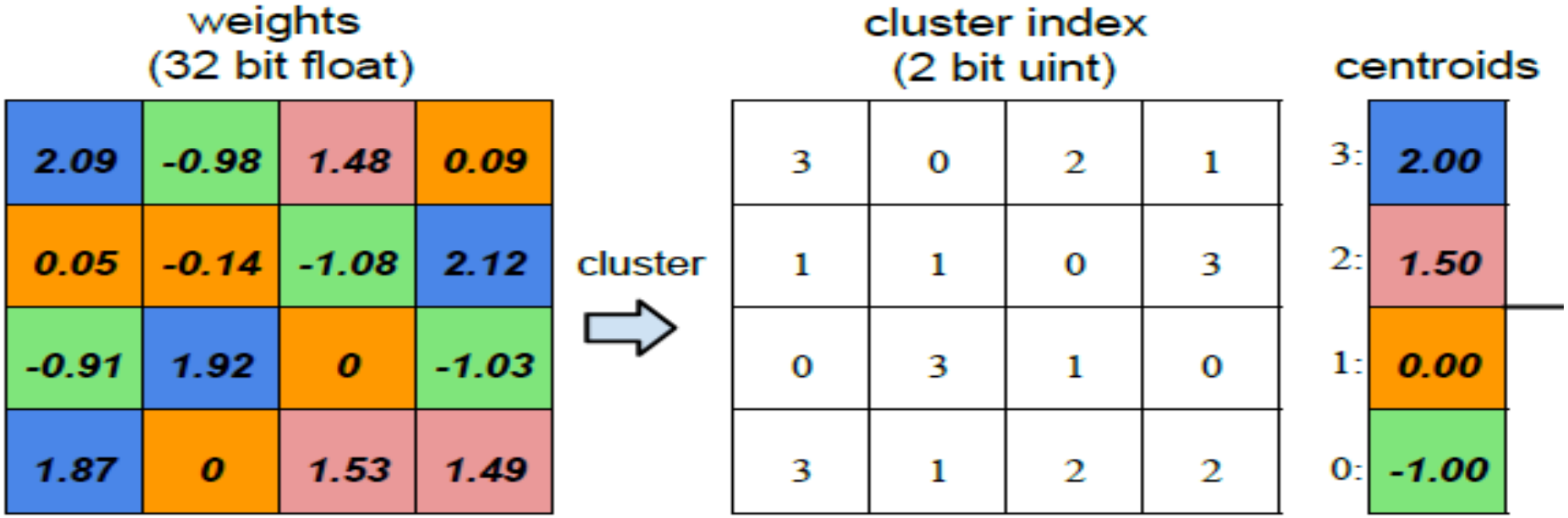
- Use k-means clustering to identify the shared weights for each layer of a trained network, so that all the weights that fall into the same cluster will share the same weight.
- The centroids of the one-dimensional k-means clustering are the shared weights.
- Weights are not shared across layers.

Quantization

- It reduces the number of bits required to represent each weight.
- Limit the number of effective weights we need to store by having multiple connections share the same weight.
- Then fine-tune those shared weights.

An Example

- Assume a 4X4 matrix.
- Weights are quantized to 4 bins (denoted with 4 colors), weights in the same bin share the same value.
- For each weight, we need to store only a small index into a table of shared weights.



Han, S., et al. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv:1510.00149.*

Pruning results on AlexNet

- For pruned AlexNet, they quantize to
  - 8-bits (256 shared weights) for each CONV layer
  - 5-bits (32 shared weights) for each FC layer
  - without any loss of accuracy

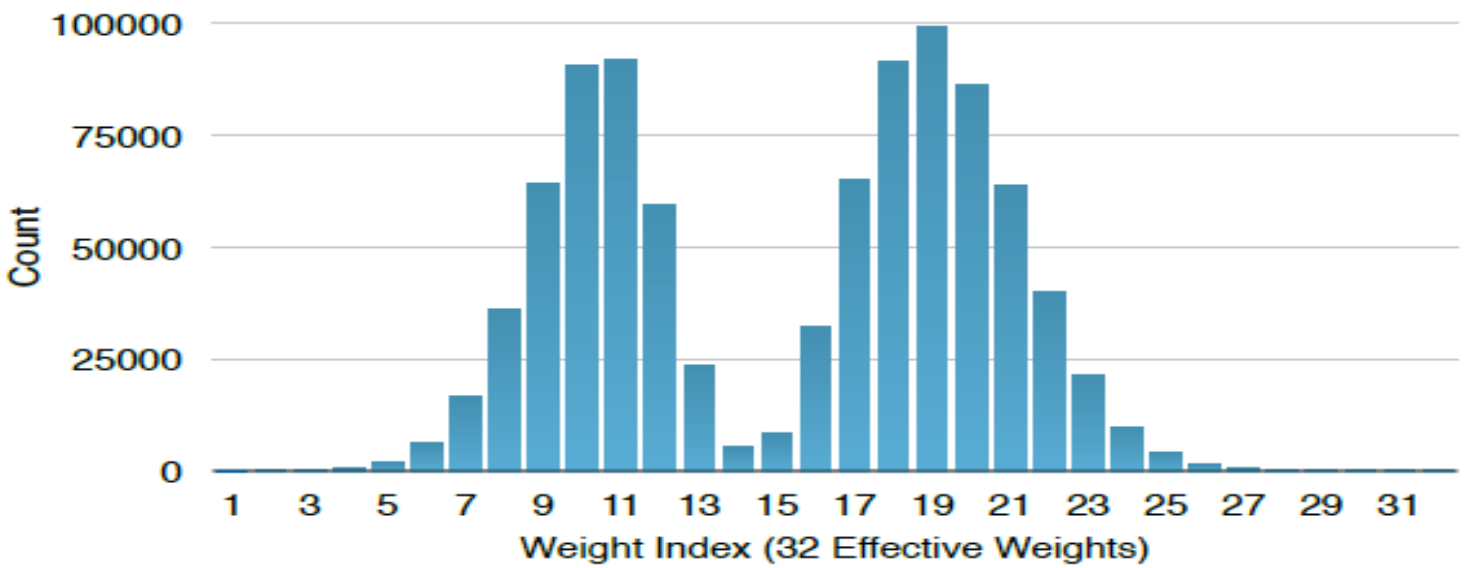
CONV = Convolution, FC = Fully Connected

Compression rate

- Assume M connections (weights), each represented by Q bits. Original network requires MQ bits.
- Compressed network has k clusters. It needs
  - M*log2(k) bits for storing indices.
  - kQ bits for storing the shared weights.
- Compression rate = MQ/(M*log2(k) + kQ)
- In our example, M=16, k=4, Q =32,
  - Compression rate = (16*32)/(16*2+4*32) = 3.2

Huffman Encoding

Third Compression Technique



- Most of the quantized weights are distributed around the two peaks.
- Huffman coding these values saves 20% - 30% of network storage.

Han, S., et al. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv:1510.00149*.

Results

Results Summary

Network	Top-1 Error	Top-5 Error	Parameters	Compress Rate
LeNet-300-100 Ref	1.64%	-	1070 KB	
LeNet-300-100 Compressed	1.58%	-	**27 KB**	**40×**
LeNet-5 Ref	0.80%	-	1720 KB	
LeNet-5 Compressed	0.74%	-	**44 KB**	**39×**
AlexNet Ref	42.78%	19.73%	240 MB	
AlexNet Compressed	42.78%	19.70%	**6.9 MB**	**35×**
VGG-16 Ref	31.50%	11.32%	552 MB	
VGG-16 Compressed	31.17%	10.91%	**11.3 MB**	**49×**

With all three techniques included, they get ~50x compression.

Pruning helps by 13x, quantization by an additional 2x, and Huffman coding by an additional 1.5x.
Final size (<12MB) is small enough to fit on a single chip.

Han, S., et al. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv:1510.00149*.

References

- "Learning both Weights and Connections for Efficient Neural Network", S. Han et al., Proceedings of NIPS, 2015
- "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding", S. Han et al., ICLR, 2016
- "EIE: Efficient Inference Engine on Compressed Deep Neural Network", S. Han et al., Proceedings of ISCA, 2016